

Projeto - Simulador de Tráfego Urbano em C

Sistemas Operacionais - Concorrência, Sincronização e Deadlocks

Linguagem obrigatória: C | Biblioteca: Pthreads | Interface: Terminal/ASCII

1. Objetivo

Desenvolver uma simulação concorrente de tráfego urbano em C, na qual veículos são representados por threads e competem por espaços de uma malha viária. O foco do trabalho é aplicar conceitos de concorrência, exclusão mútua, espera bloqueante, semáforos, variáveis de condição e prevenção de deadlocks.

Ideia central: o projeto deve mostrar, na prática, que veículos concorrentes precisam coordenar o acesso a recursos compartilhados como células da pista, cruzamentos e semáforos.

2. Regras gerais

- Equipe $\approx$ 5 integrantes
- O projeto deve ser desenvolvido obrigatoriamente em C.
- O uso de threads com `pthread_create` e `pthread_join` é obrigatório.
- O uso de mecanismos de sincronização é obrigatório: mutex, semáforos e/ou variáveis de condição.
- Não é permitido usar espera ocupada para carros parados no sinal vermelho ou aguardando liberação de cruzamento.
- O código deve ser versionado em Git/GitHub, com commits identificando a participação dos integrantes.
- O projeto deve usar visualização em terminal/ASCII.

3. Cenário da simulação

A simulação deve representar uma pequena malha urbana formada por ruas horizontais e verticais. As ruas devem conter cruzamentos, semáforos e veículos se movendo ao longo do tempo.

3.1 Requisitos mínimos do mapa

- A malha deve possuir pelo menos 8 cruzamentos.
- As vias não podem ser apenas uma cruz isolada; devem formar ruas contínuas passando por cruzamentos.
- Deve existir pelo menos uma via de mão única.
- Pode haver vias de mão dupla, desde que a ocupação de cada faixa seja controlada corretamente.
- O mapa deve ser representado por uma matriz ou estrutura equivalente.

3.2 Veículos

- A simulação deve ter entre 10 e 20 carros rodando simultaneamente.
- Cada carro deve ser uma thread.
- Deve existir pelo menos uma ambulância, também representada por uma thread.
- Cada veículo deve possuir identificador, posição atual, direção, velocidade e rota.
- Os veículos devem respeitar a direção da via e não podem atravessar paredes ou sair do mapa.

3.3 Velocidades

O movimento deve seguir um relógio global discreto, em ticks:

- Carro rápido: move a cada 1 tick.
- Carro medio: move a cada 2 ticks.
- Carro lento: move a cada 4 ticks.

4. Leis da simulação

4.1 Impenetrabilidade

- Dois veículos não podem ocupar a mesma célula da mesma faixa ao mesmo tempo.
- Um veículo só pode entrar em uma célula se ela estiver livre.
- A verificação e a ocupação da célula devem ser feitas com sincronização.

4.2 Sem teletransporte

- Um veículo só pode se mover para uma célula adjacente válida.
- Um veículo não pode pular outro veículo à frente.
- Em vias de mão única, não deve haver ultrapassagem.

4.3 Relógio global

- O relógio global deve coordenar o avanço da simulação.
- A cada tick, a thread relógio acorda as threads que podem tentar se mover.
- Os carros não devem usar loops infinitos testando o tempo continuamente.

5. Semafóros de transito

- Cada cruzamento deve possuir controle de sinal para as vias que passam por ele.
- Quando o sinal estiver vermelho para uma via, o carro deve bloquear e não consumir CPU enquanto espera.
- Quando o sinal ficar verde, as threads bloqueadas naquela condição podem ser acordadas.
- A transição do sinal deve ser segura: nenhum carro deve atravessar durante uma mudança inconsistente de estado.

6. Ambulância

- A ambulância deve ter prioridade ao chegar a um cruzamento.
- Quando a ambulância solicitar passagem, o sistema deve tornar verde a direção necessária assim que for seguro.
- A prioridade da ambulância não pode violar a regra de que dois veículos não ocupam a mesma célula.
- O sistema deve registrar visualmente ou em log quando a ambulância solicitar prioridade.

7. Concorrência e sincronização obrigatórias

O projeto deve demonstrar claramente o uso de sincronização em C. No relatório, a equipe deve explicar onde cada mecanismo foi usado.

Mecanismo	Uso esperado	Exemplo no projeto
Mutex	Proteger dados compartilhados	Mapa, células, estado dos cruzamentos
Variável de condição	Bloquear e acordar threads por eventos	Carro esperando sinal verde ou tick do relógio
Semaforo	Controlar permissão ou quantidade de recursos	Cruzamento com capacidade limitada ou controle de entrada
Thread	Executar entidades concorrentes	Carros, ambulância e relógio global

8. Deadlocks

O projeto deve evitar deadlocks. Como vários veículos podem precisar de mais de uma célula ou de um cruzamento, a equipe deve definir uma estratégia de prevenção.

A equipe deve explicar qual estratégia foi usada para evitar deadlocks. Exemplos de estratégias possíveis:

- Definir uma ordem fixa de aquisição de recursos
- Controlar a entrada nos cruzamentos
- Evitar que um veículo fique segurando um recurso enquanto espera outro indefinidamente.

9. Visualização

A simulação deve ser visual. Logs podem existir, mas não substituem a visualização.

- Terminal/ASCII, mostrando ruas, cruzamentos, semáforos e carros.
- A visualização deve permitir observar carros se movendo, semáforos mudando e ambulância recebendo prioridade.
- A cada tick, a tela deve ser atualizada para mostrar o novo estado da simulação.

10. Entregáveis

- Código-fonte em C.
- README.md com instruções de compilação e execução.
- Relatório curto explicando o que foi utilizado e como foi utilizado no trabalho
- O relatório deve explicar as principais decisões de implementação, especialmente:
 - mapa, threads, mecanismos de sincronização, ausência de espera ocupada e estratégia contra deadlock.
- Link do repositório Git/GitHub com histórico de commits.
- Lista de integrantes e responsabilidades.

11. Critérios de avaliação

Critério	Pontos	O que será observado
Corretude da simulação	2,0	Movimento válido, sem teletransporte e respeito às vias.
Visualização	2,0	Mapa visível, carros, semáforos e ambulância identificáveis.
Exclusão mútua	1,5	Dois veículos não ocupam a mesma célula indevidamente.
Sincronização de sinais	1,5	Carros param no vermelho e dormem sem consumir CPU.
Ambulância	1,0	Prioridade implementada sem quebrar as regras da simulação.
Ausência de deadlock	2,0	Estratégia clara para evitar travamentos.